

Lecture Notes: Asymptotic Analysis - Time and Space Complexity

1. Introduction: The Purpose of Data Structures & Algorithms (DSA)

Before we analyze algorithms, let's first understand why we study **Data Structures and Algorithms (DSA)**. In essence, DSA is about writing efficient programs.

- **Data Structures:** A specialized format for storing, managing, and accessing data efficiently. The way we organize our data has a massive impact on how well our programs perform.
- **Algorithms:** A sequence of well-defined steps or instructions used to process data effectively and solve a specific problem.

The primary goal is to build a good system, which can be defined by two key characteristics:

- **Efficient Time Management** (Faster Data Access & Processing)
- **Efficient Memory Management** (Efficient Data Storage)

Consider a real-world analogy with phones:

- **Phone 1:** Has a very large storage capacity but is slow to respond.
- **Phone 2:** Has less storage but has a much faster response time.

Most users would prefer Phone 2. However, our goal in DSA is to achieve the best of both worlds: programs that are both fast and memory-conscious.

2. Types of Data Structures

Data structures are broadly classified into two categories:

a. Linear Data Structures

These structures organize data in a **sequential manner**, one after the other. They are generally simpler and are useful for limited or straightforward datasets.

Examples: Arrays, Strings, Linked Lists, Stacks, Queues.

b. Non-Linear Data Structures

These structures organize data in a **hierarchical or networked manner**. They are essential when dealing with large datasets that have complex **interconnections** between data points.

Examples: Trees, Graphs, Tries, Heaps.

3. Measuring Efficiency: Why We Need Asymptotic Analysis

How do we prove that one algorithm is better than another? Let's consider a practical problem.

Problem: Calculate the sum of the first 'N' natural numbers.

Input N: 1,000,000,000 (10^9)

Let's imagine two programmers, A and B, are given this task.

> Programmer A's Approach (Loop):

C++:

```
long long sum = 0;
for (long long i = 1; i <= n; i++) {
    sum += i;
}
```

> Programmer B's Approach (Mathematical Formula):

C++:

```
long long sum = n * (n + 1) / 2;
```

To evaluate these solutions, we use a **standard benchmark**: a typical computer can perform roughly **10^8 (100 million) operations per second**.

- **Analysis of A:** The loop runs 'n' times. For $N = 10^9$, this code will perform 10^9 operations. This is greater than the 10^8 operations/sec limit. The program will take too long and result in a **TLE (Time Limit Exceeded)** error in many systems.
- **Analysis of B:** This approach involves just one multiplication, one addition, and one division—a constant number of operations. It will be almost instantaneous.

This example shows that we need a formal mathematical way to describe an algorithm's efficiency as its input (N) grows. This formal method is called **Asymptotic Analysis**.

Asymptotic Analysis is the standard mathematical representation for measuring the efficiency of an algorithm, focusing on how its performance scales with the input size. It helps us understand two key metrics:

1. **Time Complexity**: The amount of time an algorithm takes to run (total no. of operations performed) as a function of the input size.
2. **Space Complexity**: The amount of extra memory an algorithm requires as a function of the input size.

4. Asymptotic Notations: The Language of Efficiency

We use three primary mathematical notations to describe the complexity of an algorithm.

- **Big-O Notation (O)**: Describes the **Upper Bound** or the **Worst-Case** time complexity. This is the most important notation as it guarantees that the program's performance will not be worse than a certain limit.
- **Big-Omega Notation (Ω)**: Describes the **Lower Bound** or the **Best-Case** time complexity.
- **Big-Theta Notation (Θ)**: Describes the **Average-Case** time complexity, providing a **tight bound** on the algorithm's performance.

Hierarchy of Common Complexities:

Here is a list of common Big-O complexities, ordered from most efficient (fastest) to least efficient (slowest).

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

(Constant) < (Logarithmic) < (Linear) < (Log-Linear) < (Quadratic) < (Exponential) < (Factorial)

- **Intuitive Explanations:**
 - $O(1)$: **Constant**. The runtime(**total no. of operations**) does not change, no matter the size of the input.
 - $O(\log n)$: **Logarithmic**. The runtime(**total no. of operations**) grows very slowly as the input size increases. Very efficient for large datasets.
 - $O(n)$: **Linear**. The runtime(**total no. of operations**) grows directly proportional to the input size.

- $O(n^2)$: **Quadratic**. The runtime(**total no. of operations**) grows exponentially. Becomes slow very quickly (e.g., a loop inside a loop).
- $O(2^n)$: **Exponential**. The runtime(**total no. of operations**) doubles with each new element in the input. Becomes unusable even for moderately small input sizes.
- $O(n!)$: **Factorial**. The runtime(**total no. of operations**) grows extremely fast. Only practical for very small n .

Growth Rate Comparison

Let's see how fast these functions grow with $n = 10$.

Complexity	Function	Calculation for $n=10$	Result (Operations)
$O(\log n)$	$\log_{10}(n)$	$\log_{10}(10)$	1
$O(n)$	n	10	10
$O(n \log n)$	$n * \log_{10}(n)$	$10 * \log_{10}(10)$	10
$O(n^2)$	$n * n$	$10 * 10$	100
$O(2^n)$	2^n	2^{10}	1,024
$O(n!)$	$n!$	$10 * 9 * \dots * 1$	3,628,800

As you can see, the difference in growth is enormous. This is why understanding complexity is crucial for writing scalable code.

5. Analyzing Code Complexity: Practical Examples

A. Time Complexity Analysis

Case 1: Single Loops - $O(n)$

A simple loop that iterates through all n elements of an input performs n operations.

C++:

// Function to print all elements of an array

```
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << endl;
    }
}
```

- Time Complexity: $O(n)$ because the loop runs n times (no. of operations performed is N).
- Space Complexity: $O(1)$ because we only use a few variables (i), not extra space that depends on n .

Case 2: Sequential Loops - $O(n + k)$

When loops are one after another (not nested), we add their complexities.

C++:

```
void sequentialLoops(int n, int k) {
    // This loop runs 'n' times ( performs 'n' operations)
    for (int i = 0; i < n; i++) {
        print();
    }
    // This loop runs 'k' times (performs 'k' operations
    for (int j = 0; j < k; j++) {
        print();
    }
}
```

- **Time Complexity:** $O(n + k)$. If n is much larger than k , we can simplify this to $O(n)$.

Thumb Rule: Dominant Complexity

When an algorithm performs multiple, sequential operations, its overall time complexity is determined by the most expensive (slowest) operation.

Note: In a sequence of operations, always take the worst-case complexity as the final complexity because it dominates all others.

Example:

Imagine a function that first sorts an array and then prints all its elements.

Sorting (using an efficient algorithm like Merge Sort): $O(n \log n)$

Printing the array: $O(n)$

The total complexity isn't $O(n \log n + n)$. As n gets very large, the $n \log n$ term grows much faster than the n term, making the $+ n$ part insignificant.

Final Complexity: $O(n \log n)$

Case 3: Nested Loops - $O(n * k)$

When one loop is inside another, we multiply their complexities.

C++:

```
void nestedLoops(int n, int k) {
    // Outer loop runs 'n' times (performing 'n' operations)

    for (int i = 1; i <= n; i++) {
        // Inner loop runs 'k' times for EACH outer iteration ( 'k' no. of operations for each 'i' )
        for (int j = 1; j <= k; j++) {
            print(); // This line runs n * k times ( total n*k no. of operations)
        }
    }
}
```

- **Time Complexity:** $O(n * k)$.
- **Scenario A (k is a small constant):** If $n = 10^6$ and $k = 5$.

Total operations = $10^6 * 5 = 5,000,000$ ($5 * 10^6$).

This is less than 10^8 , so it will pass. In this case, k is insignificant, and we can approximate the complexity as $O(n)$.

- **Scenario B (k is large):** If $n = 10^6$ and $k = 10^3$.

Total operations = $10^6 * 10^3 = 10^9$.

This is greater than 10^8 , so it will likely cause a **TLE**. Here, k is significant and cannot be ignored. The complexity remains $O(n * k)$.

Case 4: Recursive Algorithms - $O(2^n)$

Recursive functions that make multiple calls can lead to exponential complexity. A classic example is calculating the Fibonacci sequence.

C++:

```
#include <iostream>
using namespace std;

int fib(int n) {
    // Base case
    if (n <= 1)
        return n;
    // Recursive step: makes two calls for each step
    return fib(n - 1) + fib(n - 2);
}
```

Analysis: To calculate `fib(5)`, the function calls `fib(4)` and `fib(3)`. Then, `fib(4)` calls `fib(3)` and `fib(2)`, and so on.



Time Complexity: $O(2^n)$. The number of function calls roughly doubles for every increase in `n`, leading to an explosive growth in operations.

B. Space Complexity Analysis

Space complexity measures the **extra space** or temporary memory an algorithm uses, not including the space taken by the inputs themselves. The key question is: "Does the memory I'm using grow as the input `n` grows?"

When Space is $O(1)$ (Constant):

The space used does not depend on the input size. This typically includes simple variables used for iteration or storing temporary values.

Example: `int i`, `int temp`, `int maxVal`. No matter if `n` is 10 or 10 million, you are still only creating one integer variable for each.

When Space is $O(n)$ (Linear) or higher:

The space used is directly proportional to the input size. This almost always happens when you create a data structure whose size depends on `n`.

Example: `int newArray[n]`, `vector<int> temp(n)`, a recursive function call stack that goes `n` levels deep. If `n` doubles, the memory required for your new data structure also doubles.

Case 1: Constant Space - $O(1)$

The algorithm uses a fixed amount of memory, regardless of the input size.

C++:

```
// Function to find the maximum value in an array
```

```
void findMax(int arr[], int n) {
    // We only use two extra variables: maxVal and i.
    // Their memory usage does not depend on 'n'.
```

```

int maxVal = arr[0];

for (int i = 1; i < n; i++) {
    if (arr[i] > maxVal) {
        maxVal = arr[i];
    }
}

cout << maxVal << endl;
}

```

Time Complexity: $O(n)$ (due to the 'n' no. of iterations).

Space Complexity: $O(1)$ (only 'maxVal' and 'i' are created).

Case 2: Linear Space - $O(n)$

The memory required grows linearly with the input size. This often happens when you create an auxiliary data structure, like another array.

C++:

// A function that creates a temporary copy of an array

```

void createCopy(int arr[], int n) {
    // This creates a new array of size 'n'.
    // The space needed depends directly on the input size 'n'.

    int temp[n];

    for (int i = 0; i < n; i++) {
        temp[i] = arr[i];
    }
}

```

Time Complexity: $O(n)$ (due to the 'n' no. of iterations).

Space Complexity: $O(n)$ (due to the 'temp' array).

6. Homework Exercise:

- Create a table or spreadsheet.
- Calculate the values of 2^n and $n!$ for 'n' from 1 to 20.

- Observe at what point $n!$ overtakes 2^n and how rapidly both values become astronomically large. This will give you a tangible feel for why these complexities are generally avoided for large inputs.